chiselWare

Standard

For Chisel-based Semiconductor IP Cores

Version 0.9.0

November 4, 2026

https://chiselware.org

Copyright © 2026 Rocksavage Technology, Inc. All Rights Reserved

Notice

This document is a working draft of the chiselWare Standard (CWS). It is published for review and comment purposes only. The contents of this document are subject to change without notice.

Copyright © 2026 Rocksavage Technology, Inc. All rights reserved.

No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of Rocksavage Technology, Inc., except for the purpose of review and comment during the standards development process.

chiselWare, the chiselWare logo, and IP Factory are trademarks of Rocksavage Technology, Inc. All other trademarks are the property of their respective owners.

Inquires can be made to:

Rocksavage Technology, Inc.
1821 S. Bascom Avenue, Suite 374
Campbell, CA 95008

Email: admin@chiselware.org

Abstract

This standard defines the mandatory requirements for chiselWare-compliant semiconductor IP cores developed using the Chisel hardware description language. It specifies requirements for developer registration, directory structure, versioning, parameterization, coding style, verification, documentation, and regression testing.

Conformance to this standard is a necessary and sufficient condition for chiselWare Certification and authorized use of the chiselWare trademark and logo. The methods by which conformance is established are defined and administered by the chiselWare Administrator and may include automated rule checking, static analysis, manual review, or any combination thereof.

Keywords: chiselWare, Chisel, semiconductor IP, hardware description language, IP reuse, certification, Scala, FPGA, ASIC.

Revision History

Version	Date	Description
0.9.0	November 4, 2026	New rules
0.8.2	July 20, 2026	Authorship policy
0.8.0	June 4, 2026	Add formal verification support
0.7.5	May 17, 2026	Update to align with CWDG
0.7.4	April 25, 2026	Initial release

Contents

1 Scope **3**

2 Purpose **4**

3 Terms, Definitions, and Abbreviations **5**
- 3.1 Verbal Forms . . . 5
- 3.2 Roles . . . 5
  - 3.2.1 Core Developer . . . 5
  - 3.2.2 Core User . . . 5
  - 3.2.3 chiselWare Administrator . . . 5
- 3.3 Artifacts and Registry . . . 6
  - 3.3.1 Developer ID . . . 6
  - 3.3.2 chiselWare Developer Agreement . . . 6
  - 3.3.3 Organization . . . 6
  - 3.3.4 Organization ID . . . 6
  - 3.3.5 Team . . . 6
  - 3.3.6 Team ID . . . 7
  - 3.3.7 Chisel Core . . . 7
  - 3.3.8 chiselWare Core . . . 7
  - 3.3.9 chiselWare Core Registry . . . 7
- 3.4 Compliance and Certification . . . 7
  - 3.4.1 chiselWare Certification . . . 7
  - 3.4.2 Certification Version . . . 7
  - 3.4.3 Automated Design Review (ADR) . . . 7
  - 3.4.4 Static Compliance Check . . . 8
- 3.5 Technical Terms . . . 8
  - 3.5.1 Standard Software Environment (SSE) . . . 8
  - 3.5.2 Semantic Version . . . 8
  - 3.5.3 Abbreviations . . . 8

4 Normative References **9**

5 Informative References **10**

6 Conformance **11**
- 6.1 Certification Prerequisites . . . 11
- 6.2 Certification Process . . . 11

6.3 Certification Version and Stability . . . 12
6.4 Trademark and Logo Authorization . . . 12
6.5 Conformance Checklist . . . 13

7 Directory Structure **14**
7.1 General . . . 14
7.2 Repository Root . . . 14
7.3 Module Directory . . . 15
7.4 Source Directory Structure . . . 16
7.5 Package Namespace . . . 16
7.6 Required Source Files . . . 16
7.7 Documentation Directory . . . 17
7.8 Standard Software Environment Constraint . . . 17

8 Versioning **18**
8.1 General . . . 18
8.2 Version Position Semantics . . . 18
8.3 Version Reset Rules . . . 18
8.4 Pre-Release Versions . . . 19
8.5 Version Tagging . . . 19
8.6 Certification and Versioning . . . 19

9 Parameters **20**
9.1 General . . . 20
9.2 Top-Level Parameter Case Class . . . 20
9.3 Parameter Validation . . . 20
9.4 Core Instantiation . . . 21
9.5 Internal Parameterization . . . 21

10 Coding Style **22**
10.1 General . . . 22
10.1.1 Scala Coding Style . . . 22
10.1.2 Chisel Coding Style . . . 22
10.2 chiselWare Coding Style . . . 22
10.2.1 Precedence . . . 22
10.2.2 Scripting and Build Languages . . . 23
10.2.3 Hardware Description Languages . . . 23
10.2.4 Scaladoc Requirements . . . 23

11 Verification **24**
11.1 General . . . 24
11.2 Directed Tests . . . 24
11.3 Constrained-Random Verification . . . 24
11.4 Coverage Requirements . . . 24
11.4.1 Configuration Coverage . . . 25
11.4.2 Test Documentation . . . 25
11.5 Formal Verification . . . 25
11.5.1 General . . . 25

Copyright © 2026 Rocksavage Technology, Inc.

11.5.2 Formal Testbench Structure . . . 25
11.5.3 Formal Test Runner . . . 27
11.5.4 ADR Checking . . . 27
11.6 Code Coverage . . . 27
11.6.1 General . . . 27
11.6.2 Coverage Reporting . . . 28
11.6.3 Coverage Exclusions . . . 28

12 Documentation **29**
12.1 General . . . 29
12.2 Scaladoc Requirements . . . 29
12.2.1 General . . . 29
12.2.2 Publication . . . 29
12.3 User Guide . . . 29
12.3.1 General . . . 29
12.3.2 Required Sections . . . 30
12.3.3 Version Consistency . . . 30
12.3.4 Port Description Consistency . . . 30
12.3.5 Parameter Description Consistency . . . 30
12.3.6 Generated Formats . . . 31
12.4 README . . . 31
12.4.1 General . . . 31
12.4.2 Required Content . . . 31
12.5 License . . . 31
12.5.1 Permitted License Terms . . . 31
12.5.2 Source File Headers . . . 33
12.5.3 License Consistency . . . 33

13 Regressions and CI/CD **34**
13.1 General . . . 34
13.2 Makefile . . . 34
13.2.1 Required Make Targets . . . 34
13.3 Regression Sequence . . . 35
13.4 Regression Pass/Fail Criterion . . . 35
13.5 Synthesis Requirements . . . 36
13.5.1 General . . . 36
13.5.2 Required Configurations . . . 36
13.5.3 Synthesis Deliverables . . . 36
13.5.4 Timing Closure . . . 37
13.6 IP Factory Deliverables . . . 37
13.6.1 General . . . 37
13.6.2 Required IP Factory Deliverables . . . 37
13.7 GitHub Actions Workflow . . . 37

14 Licensing and Provenance **39**
14.1 Scope . . . 39
14.2 Policy of Original Authorship . . . 39
14.3 Prohibited Provenance . . . 39

Copyright © 2026 Rocksavage Technology, Inc.

14.4 Permitted Provenance . . . 40
14.5 Attribution Requirements . . . 40
14.6 Relationship to Certification . . . 40

Annexes **42**

A Conformance Checklist (Normative) **42**
A.1 General . . . 42
A.2 Scala Coding Style . . . 42
A.2.1 SCALA-001 . . . 42
A.2.2 SCALA-002 . . . 43
A.2.3 SCALA-019 . . . 43
A.2.4 SCALA-020 . . . 43
A.2.5 SCALA-021 . . . 43
A.2.6 SCALA-022 . . . 43
A.2.7 SCALA-023 . . . 43
A.2.8 SCALA-024 . . . 44
A.2.9 SCALA-025 . . . 44
A.2.10 SCALA-026 . . . 44
A.2.11 SCALA-029 . . . 44
A.3 Chisel Coding Style . . . 44
A.3.1 CHISEL-001 . . . 44
A.3.2 CHISEL-002 . . . 45
A.3.3 CHISEL-003 . . . 45
A.3.4 CHISEL-004 . . . 45
A.3.5 CHISEL-005 . . . 45
A.4 chiselWare Guidelines . . . 45
A.4.1 CW-001 . . . 45
A.4.2 CW-007 . . . 45
A.4.3 CW-008 . . . 46
A.4.4 CW-009 . . . 46
A.4.5 CW-010 . . . 46
A.4.6 CW-011 . . . 46
A.4.7 CW-012 . . . 46
A.4.8 CW-013 . . . 47
A.5 Code Coverage . . . 47
A.5.1 COV-001 . . . 47
A.6 Synthesis Deliverables . . . 48
A.6.1 SYN-001 . . . 48
A.6.2 SYN-002 . . . 48
A.6.3 SYN-009 . . . 48
A.7 Documentation . . . 48
A.7.1 DOC-001 . . . 48
A.7.2 DOC-002 . . . 48
A.7.3 DOC-003 . . . 49
A.7.4 DOC-004 . . . 49
A.7.5 DOC-005 . . . 49

Copyright © 2026 Rocksavage Technology, Inc.

A.7.6 DOC-006 49

B Relationship to the chiselWare Developers Guide (Informative) 50

C Standard Software Environment (Normative) 51
C.1 General 51
C.2 Rationale 51
C.3 Toolchain Specification 52
C.3.1 Docker Image 52
C.3.2 Azure Virtual Machine 53
C.4 Self-Built Environments 53
C.5 SSE Updates 53

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 1

Scope

This standard defines the requirements for chiselWare®-compliant semiconductor IP cores developed using the Chisel hardware description language. It specifies mandatory requirements for directory structure, versioning, parameterization, coding style, verification, documentation, and regression testing.

This standard applies to all IP cores submitted for inclusion in the chiselWare core library. It does not prescribe the internal microarchitecture or functional behavior of any core, nor does it constrain the choice of algorithms or design techniques employed by the core developer.

This standard does not apply to:

- Cores developed using hardware description languages other than Chisel
- Software components that accompany a chiselWare core unless those components are included as required deliverables defined herein

All scripting and build automation associated with a chiselWare core shall use only tools that are part of the chiselWare Standard Software Environment (SSE), such as Bash and Make. No other scripting or build languages are permitted.

Chapter 2

Purpose

The purpose of this standard is to establish a uniform, enforceable methodology for the development of semiconductor IP cores within the chiselWare ecosystem. Uniformity across cores enables the systematic, scalable, and reliable mechanical verification of conformance — replacing ad hoc peer review with automated checking that can be applied consistently across a growing library of cores from a distributed developer community.

A secondary purpose is to provide users of chiselWare cores with confident expectations of quality, completeness, and consistency regardless of which core or which developer organization produced the core.

This standard is intended to evolve. Requirements are added deliberately and with clear justification. Requirements that cannot be automatically checked are highly discourgaged as candidates for admission to this standard.

Chapter 3

Terms, Definitions, and Abbreviations

3.1 Verbal Forms

The following verbal forms are used throughout this standard in accordance with ISO/IEC Directives, Part 2:

- **shall** — indicates a mandatory requirement. Deviation constitutes non-conformance.
- **should** — indicates a recommendation. Deviation is permitted with justification.
- **may** — indicates a permissible action. No preference is implied.

3.2 Roles

3.2.1 Core Developer

An individual who has completed registration at `chiselware.org`, received a Developer ID, and executed a chiselWare Developer Agreement. Only registered Core Developers may submit cores for chiselWare Certification. A Core Developer operates within the context of at least one Organization and one Team.

3.2.2 Core User

The individual or organization that integrates a chiselWare Core into a semiconductor design.

3.2.3 chiselWare Administrator

The organizational role responsible for maintaining this standard, operating `chiselware.org`, administering the Developer registration process, managing the chiselWare Core Registry, and authorizing use of the chiselWare trademark and logo. The chiselWare Administrator role may be held by a commercial entity, a nonprofit organization, or a designated governance body.

3.3 Artifacts and Registry

3.3.1 Developer ID

A unique identifier assigned by the chiselWare Administrator to a Core Developer upon successful registration at `chiselware.org`. The Developer ID is associated with an executed chiselWare Developer Agreement and is a prerequisite for core submission and certification.

3.3.2 chiselWare Developer Agreement

The legal agreement executed between a Core Developer and the chiselWare Administrator at the time of registration. It establishes the terms under which cores may be submitted, certified, and distributed within the chiselWare ecosystem.

3.3.3 Organization

A named entity registered at `chiselware.org` representing a company, academic institution, or independent developer group. An Organization is assigned a unique Organization ID by the chiselWare Administrator upon registration. A Core Developer may register a new Organization or join an existing one.

3.3.4 Organization ID

A two-character alphanumeric identifier in the format `[a-zA-Z0-9]{2}` assigned by the chiselWare Administrator to a registered Organization. The Organization ID forms part of the chiselWare package namespace for all cores developed under that Organization.

The chiselWare Administrator reserves the Organization ID `IN` for the Independent Organization, a catchall organization provided for Core Developers who have no formal institutional affiliation. Core Developers who register under the Independent Organization (`oIN`) are assigned their own Team ID and operate independently. Membership in the Independent Organization does not imply any relationship or affiliation between its member Teams.

A Core Developer may be associated with more than one Organization simultaneously. For example, a developer employed by a university may contribute cores under that university's Organization ID as part of a sponsored research project, while also contributing independently developed cores under the Independent Organization (`oIN`). Cores developed under `oIN` are solely the work of the individual Core Developer and carry no implied affiliation with any other Organization with which that developer may be associated. It is the sole responsibility of the Core Developer to ensure that cores submitted under `oIN` do not contain intellectual property belonging to any other Organization.

A core developed under the Independent Organization will have a package namespace of the form:

```
org.chiselware.cores.oIN.t<TeamID>.<corename>
```

3.3.5 Team

A named group of Core Developers operating within a registered Organization. A Team is assigned a unique Team ID by the chiselWare Administrator upon registration. A Core Developer may register a new Team within an Organization or join an existing one.

Copyright © 2026 Rocksavage Technology, Inc.

3.3.6 Team ID

A three-character alphanumeric identifier in the format `[a-zA-Z0-9]{3}` assigned by the chiselWare Administrator to a registered Team. The Team ID forms part of the chiselWare package namespace for all cores developed under that Team.

3.3.7 Chisel Core

An IP core implemented in the Chisel hardware description language that has not achieved chiselWare Certification. A Chisel Core may conform to some or all requirements of this standard but shall not be represented as a chiselWare Core and shall not bear the chiselWare logo or trademark.

3.3.8 chiselWare Core

An IP core implemented in the Chisel hardware description language that has achieved chiselWare Certification under this standard. Only chiselWare Cores may bear the chiselWare logo and be represented as chiselWare in marketing materials, documentation, or product listings.

3.3.9 chiselWare Core Registry

The authoritative database maintained by the chiselWare Administrator recording all chiselWare Cores, their certification status, their Certification Version, and their associated Organization and Team IDs.

3.4 Compliance and Certification

3.4.1 chiselWare Certification

The formal determination by the chiselWare Administrator that a core satisfies all requirements of this standard under a specific Certification Version. Certification requires successful completion of both Automated Design Review and all Static Compliance Checks defined in Annex A.

3.4.2 Certification Version

The version of this standard against which a chiselWare Core was certified. A core certified under a given Certification Version remains certified under that version in perpetuity, regardless of subsequent revisions to this standard.

3.4.3 Automated Design Review (ADR)

The rule-based conformance checking system operated by the chiselWare Administrator that verifies submitted cores against the requirements of this standard. The ADR system employs a combination of deterministic static analysis tools and large language model (LLM) based analysis to evaluate conformance across the rule sets defined in Annex A.

Deterministic checks — including those performed by `scalafmt` and `scalafix` — produce definitive pass/fail results. LLM-based checks apply trained judgment to evaluate conformance requirements that are not amenable to purely deterministic analysis, such as documentation quality, narrative adequacy, and coding style consistency.

Copyright © 2026 Rocksavage Technology, Inc.

The chiselWare Administrator acknowledges that LLM-based analysis may produce false positives. A Core Developer who believes an ADR finding is incorrect may request a human review by the chiselWare Administrator. The chiselWare Administrator may issue a waiver for any ADR finding determined to be a false positive following human review. Waivers are recorded in the chiselWare Core Registry and are specific to the core version and ADR rule for which they were granted.

Each normative requirement in this standard has a corresponding ADR rule as defined in Annex A.

3.4.4 Static Compliance Check

A conformance check that is not performed by the ADR system but is nonetheless required for chiselWare Certification. Static Compliance Checks include source code formatting verification via `scalafmt` and lint rule verification via `scalafix`. Passing all Static Compliance Checks is a necessary condition for chiselWare Certification.

3.5 Technical Terms

3.5.1 Standard Software Environment (SSE)

The versioned, containerized development environment specified by the chiselWare Administrator that defines the exact toolchain — including Chisel, Scala, sbt, simulation, synthesis, and documentation tools — against which chiselWare Cores are developed and certified. The SSE is distributed as a Docker image via a public GitHub repository: `https://github.com/chiselWare/devkit`

3.5.2 Semantic Version

A version identifier of the form `x.y.z` where `x`, `y`, and `z` are non-negative integers, used to communicate the nature of changes between releases of a chiselWare Core. The semantics of each position are defined in Section 8 of this standard.

3.5.3 Abbreviations

Abbreviation	Expansion
ADR	Automated Design Review
CI/CD	Continuous Integration / Continuous Delivery
CWS	chiselWare Standard
CWDG	chiselWare Developers Guide
HDL	Hardware Description Language
IP	Intellectual Property
MAN	Manual Review
SCC	Static Compliance Check
SemVer	Semantic Versioning
SSE	Standard Software Environment

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 4

Normative References

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this standard.

Scala Style Guide
https://docs.scala-lang.org/style/

Chisel Developer Style Guide
https://www.chisel-lang.org/docs/developers/style/

Google Shell Style Guide
https://google.github.io/styleguide/shellguide.html

GNU Make Conventions
https://www.gnu.org/prep/standards/html_node/Makefile-Conventions.html

Apache License, Version 2.0
https://www.apache.org/licenses/LICENSE-2.0

Semantic Versioning 2.0.0
https://semver.org/

Chapter 5

Informative References

The following resources provide essential background information relevant to this standard but do not constitute normative requirements. Every Core Developer should be familiar with them.

- **chiselWare Developers Guide (CWDG)** — Companion document to this standard. Provides implementation guidance, philosophy, and worked examples.
- **chiselWare Dev Kit** — https://github.com/chiselWare/devkit — Contains the chiselWare standard, Developers Guide, and Dockerfile of the SSE.
- **Template Repo** — https://github.com/chiselWare/00-000-dff — Contains a complete example of a certified chiselWare Core using a D-flip-flop as the most simplistic example possible. While not normative by definition, it is an essential starting point for new chiselWare Core development.
- **Reuse Methodology Manual, Third Edition** — Keating and Bricaud. The foundational methodology reference for semiconductor IP reuse.

Chapter 6

Conformance

Conformance to this standard is binary. A core is either a chiselWare Core or it is not. There is no partial conformance, provisional certification, or conditional certification status.

6.1 Certification Prerequisites

A core mission shall satisfy all of the following prerequisites before the certification process is initiated:

1. The submitting Core Developer shall hold a valid Developer ID and an executed chiselWare Developer Agreement.
2. The core shall be associated with a registered Organization ID and Team ID.
3. The core shall be developed operate within the Standard Software Environment for the target Certification Version.
4. The core shall satisfy the technical requirements contained in the chiselWare Standard.
5. All source code shall pass all the tests that are included in the Certification Process as found in Section 6.2.
6. A new core mission shall be certified against the current version of this standard at the time of mission. The chiselWare Administrator will not accept initial certification missions against prior versions of the standard.

6.2 Certification Process

Certification is achieved by successfully completing the following:

1. **Regression Test** — execution and successful completion of standard chiselWare tests using the standard regression Makefile.
2. **Static Compliance Checks** — automated verification of source code formatting and lint rules, that use`scalafmt` for formatting verification and `scalafix` for lint rule verification.

3. **Automated Design Review (ADR)** — systematic rule-based verification of all normative requirements defined in Sections 7 through 13 of this standard.

All checks shall pass without exception. A core that fails any regression test, ADR rule, or any Static Compliance Check shall not be certified under this standard.

6.3 Certification Version and Stability

Upon successful certification, the core is recorded in the chiselWare Core Registry with its Certification Version. The following policies apply:

1. A chiselWare Core certified under a given Certification Version remains certified under that version in perpetuity. Subsequent revisions to this standard do not retroactively invalidate an existing certification.
2. When a new version of this standard is released, previously certified cores will be regression-tested against the new version. Failures will be reported to the Core Developer and noted in the chiselWare Core Registry.
3. It is the Core Developer's prerogative to update a core to achieve certification under a newer version of this standard. Such an update constitutes a new certification and will be recorded as such in the chiselWare Core Registry.
4. A core that has not been updated to the current Certification Version remains available in the chiselWare Core Registry and retains its chiselWare Core status, but shall be marked with its Certification Version so that Core Users can make informed decisions.

6.4 Trademark and Logo Authorization

All chiselWare Core repositories are hosted within the `chiselware.org` GitHub organization and are private until chiselWare Certification is granted. Use of the chiselWare name, logo, and trademark in association with an IP core is authorized solely by chiselWare Certification. Specifically:

1. Only cores recorded as chiselWare Cores in the chiselWare Core Registry may be represented as chiselWare in any marketing material, documentation, product listing, or public communication.
2. Authorization to use the chiselWare logo is granted by the chiselWare Administrator upon certification and remains valid for the certified core version.
3. Any Chisel Core published outside the `chiselware.org` GitHub organization is by definition uncertified and shall not bear the chiselWare logo or trademark, nor be represented as chiselWare in any context, regardless of its degree of conformance to this standard. Core Developers who publish forks or derived works outside `chiselware.org` do so outside the chiselWare ecosystem and assume sole responsibility for any representations made about such cores.

Copyright © 2026 Rocksavage Technology, Inc.

6.5 Conformance Checklist

Annex A provides a complete cross-reference of every normative **shall** requirement in this standard to its corresponding verification mechanism. For each requirement, Annex A identifies whether conformance is established by Static Compliance Check, Automated Design Review, CI workflow, or manual review by the chiselWare Administrator.

Core Developers shall use Annex A as the authoritative reference for understanding what is required for chiselWare Certification and how each requirement will be verified. It is strongly recommended that Core Developers work through Annex A as a pre-mission checklist before initiating formal certification, as doing so will identify non-conformances that would otherwise result in certification failure.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 7

Directory Structure

7.1 General

All chiselWare Core repositories shall conform to the directory structure defined in this section. The directory structure is designed to be consistent with Scala and Chisel build tool conventions while extending those conventions to accommodate the full set of required IP deliverables defined in this standard.

A fully compliant template core is provided in the chiselWare template repository https://github.com/chiselWare/00-000-dff. The directory structure, build configuration, and file naming conventions of the template repository define the compliant pattern against which certification is assessed.

7.2 Repository Root

The repository root shall contain the following required files:

File	Description
`build.sbt`	sbt build definition
`Makefile`	Top-level regression harness
`README.md`	Repository overview
`LICENSE.md`	Core license terms
`AUTHORSHIP.md`	AI-assistance disclosure and incorporated Apache 2.0 source attribution, per Chapter 14
`project/plugin.sbt`	sbt plugins

The repository root shall also contain the following required directories:

Directory	Description
`modules/`	All core source, test, and documentation deliverables
`project/`	sbt project configuration files
`docs/`	Repository-level documentation
`.git`	GitHub files
`.github/`	GitHub Actions workflow definitions
`.ipf/`	Generated IP Factory deliverables

The following configuration files are required at the repository root and shall use the versions distributed as part of the Standard Software Environment for the target Certification Version:

File	Description
`.scalafmt.conf`	scalafmt formatting configuration
`.scalafix.conf`	scalafix lint configuration
`.scalafix-test.conf`	scalafix test-specific lint configuration

There are files and directories that are sometimes generated by development tools which shall be present in `.gitignore`. They are not required deliverables and shall not be committed to the repository. Some common examples are:

File or Directory	Description
`.bloop/`	Bloop build server cache
`.bsp/`	Build Server Protocol configuration
`.metals/`	Metals language server cache
`.vscode/`	VS Code configurations

No files or directories other than those defined in this section shall appear at the repository root unless explicitly permitted by the chiselWare Administrator.

7.3 Module Directory

All core source, test, and documentation deliverables shall reside under a `modules/` directory at the repository root. A repository shall contain at least one core module. Repositories containing multiple core modules shall organize each module as an independent directory under `modules/`.

The `<corename>` directory name shall be the lowercase name of the core using only characters in the set `[a-z0-9]`.

Copyright © 2026 Rocksavage Technology, Inc.

7.4 Source Directory Structure

Within each core module, the following directory structure shall be observed:

```
modules/
  <corename>/
    src/
      main/
        resources/vsrc/
        scala/org/chiselware/cores/o<OrgID>/t<TeamID>/<corename>/
          <CoreName>Params.scala
          <CoreName>.scala
      test/
        scala/org/chiselware/cores/o<OrgID>/t<TeamID>/<corename>/
          <CoreName>Tb.scala
          <CoreName>Test.scala
          <CoreName>FormalTb.scala   (optional)
          <CoreName>FormalTest.scala (optional)
    docs/
      user-guide/
        ...
    sw/ (optional)
```

The `sw/` directory is optional and shall be present only when software deliverables are associated with the core.

7.5 Package Namespace

All chiselWare Core Scala packages shall conform to the following namespace convention:

```
org.chiselware.cores.o<OrgID>.t<TeamID>.<corename>
```

Where:

- `<OrgID>` is the two-character Organization ID assigned by the chiselWare Administrator
- `<TeamID>` is the three-character Team ID assigned by the chiselWare Administrator
- `<corename>` is the lowercase core name matching the module directory name defined in Section 7

The package declaration in all Scala source files shall exactly match the namespace derived from the directory path of that source file.

7.6 Required Source Files

The main source directory shall contain at minimum the following Scala source files:

File	Description
`<CoreName>Params.scala`	Top-level parameter case class and companion object
`<CoreName>.scala`	Top-level module definition

Copyright © 2026 Rocksavage Technology, Inc.

The test source directory shall contain at minimum the following Scala source files:

File	Description
`<CoreName>Tb.scala`	Simulation testbench
`<CoreName>Test.scala`	Simulation and formal test suite

The following files are optional and should only be present if a formal test suite is provided:

File	Description
`<CoreName>FormalTb.scala`	Formal verification testbench containing properties, assumptions, assertions, and cover points
`<CoreName>FormalTest.scala`	Formal test runner using `verify()` with `BoundedCheck`

Where `<CoreName>` is the UpperCamelCase form of the core name. The `FormalTest` tag defined in `<CoreName>FormalTest.scala` allows formal tests to be run independently of simulation tests at the sbt level.

7.7 Documentation Directory

The `docs/user-guide/` directory shall contain at minimum the following:

Path	Description
`<CoreName>.tex`	Top-level LaTeX source file for the User Guide
`pdf/`	Directory for generated PDF output
`html/`	Directory for generated HTML output
`images/`	Directory for image files referenced in the User Guide

The `draw.io/` and `wavedrom/` directories are required if and only if the User Guide contains diagrams or timing diagrams generated by those tools respectively.

7.8 Standard Software Environment Constraint

All build, simulation, synthesis, and documentation tooling invoked within a chiselWare Core repository shall be confined to the Standard Software Environment defined by the chiselWare Administrator for the target Certification Version. No build or scripting tool outside the SSE shall be introduced as a dependency of a chiselWare Core.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 8

Versioning

8.1 General

All chiselWare Cores shall use Semantic Versioning as defined in this section to communicate the nature of changes between releases to Core Users. The version identifier shall be of the form `x.y.z` where `x`, `y`, and `z` are non-negative integers.

8.2 Version Position Semantics

The three positions of the version identifier carry the following mandatory semantics:

Position	Name	Semantics
`x`	Major	Incremented when a new feature or capability is added to the core
`y`	Minor	Incremented when any functional change to the RTL results in different gates being generated during synthesis
`z`	Patch	Incremented for any non-functional change to the core

Non-functional changes include but are not limited to: documentation updates, additional test cases, and corrections to comments or whitespace.

8.3 Version Reset Rules

The following reset rules shall apply when a version position is incremented:

1. When the Major version (`x`) is incremented, the Minor and Patch positions shall both be reset to zero.
2. When the Minor version (`y`) is incremented, the Patch position shall be reset to zero.
3. The Patch position (`z`) is incremented independently with no effect on other positions.

8.4 Pre-Release Versions

Versioning requirements defined in this section apply exclusively to releases tagged on the `main` branch. Commits on feature branches and the `staging` branch are internal development artifacts and are not subject to versioning requirements.

The first certified release of a chiselWare Core shall be version `1.0.0`. A Core Developer should submit a core for chiselWare Certification when it has successfully completed all required checks defined in Annex A and the Core Developer considers the core ready for production use by Core Users.

There are no constraints on how a Core Developer manages version identifiers on feature branches during development. However, the version field in `build.sbt` on a feature branch shall reflect the intended release version that the core will carry upon promotion to `main`. This ensures that the version consistency requirements defined in Section 8.5 are satisfied at the point of promotion without requiring a change to `build.sbt`.

8.5 Version Tagging

The version of a chiselWare Core shall be recorded in the following locations and shall be identical in all locations:

1. The `version` field of `build.sbt`
2. The Git tag applied to the release commit, of the form `v<x>.<y>.<z>`
3. The version field of the User Guide as defined in Section 12

Any discrepancy between these three version records shall constitute a conformance failure.

8.6 Certification and Versioning

Each chiselWare Certification is associated with a specific core version. The following rules apply:

1. A core shall be certified at a specific `x.y.z` version. Certification does not extend to any other version of the same core.
2. Every new release of a chiselWare Core shall require chiselWare Certification.
3. A core version that has not achieved chiselWare Certification shall not be represented as a chiselWare Core regardless of the nature of the changes from the previously certified version.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 9

Parameters

9.1 General

Chisel and Scala provide numerous mechanisms for parameterizing hardware designs. chiselWare preserves this flexibility within the internal implementation of a core. At the top level of a chiselWare Core, however, parameterization shall follow the conventions defined in this section to ensure that all chiselWare Cores present a consistent interface to Core Users.

9.2 Top-Level Parameter Case Class

Every chiselWare Core shall define a top-level parameter case class that aggregates all parameters that influence Verilog generation or testbench behavior. The following rules apply:

1. The case class shall be named `<CoreName>Params` where `<CoreName>` is the UpperCamelCase name of the core.
2. The case class shall be defined in the file `<CoreName>Params.scala` as specified in Section 7.
3. The case class shall reside in the package namespace defined in Section 7.
4. All parameters that influence the generated Verilog or testbench behavior shall be members of this case class. No such parameters shall be defined outside of this case class at the top level.
5. All parameters shall have default values.

9.3 Parameter Validation

All parameters that have defined bounds or constraints shall include a `require` statement within the case class body to enforce those constraints at elaboration time. The following rules apply:

1. Every `require` statement shall include an informative error message that identifies the parameter, states the violated constraint, and provides sufficient information for the Core User to correct the misconfiguration.

2. The error message shall not assume knowledge of the internal implementation of the core.
3. All parameter bounds defined in the User Guide as specified in Section 12 shall have a corresponding `require` statement. Any discrepancy between documented bounds and enforced bounds shall constitute a conformance failure.

9.4 Core Instantiation

A chiselWare Core shall be instantiated by constructing an instance of its `<CoreName>Params` case class and passing that instance to the top-level module constructor. No other instantiation pattern is permitted at the top level. The following example illustrates the required pattern:

```
val myParams = CoreNameParams(
    parameterA = valueA,
    parameterB = valueB
)
val myCore = CoreName(myParams)
```

Core Users shall not be required to pass parameters to a chiselWare Core by any mechanism other than the top-level parameter case class.

9.5 Internal Parameterization

Below the top level, Core Developers may use any Scala or Chisel parameterization mechanism at their discretion, including but not limited to: implicit parameters, type parameters, abstract members, and nested case classes. No constraints are imposed by this standard on internal parameterization patterns provided that all externally visible parameters are surfaced through the top-level case class defined in Section 9.2.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 10

Coding Style

10.1 General

chiselWare Cores shall adhere to the coding style requirements defined in this section. Coding style requirements are organized in order of precedence. In the event of a conflict between requirements at different levels, the higher precedence requirement shall govern.

10.1.1 Scala Coding Style

All Scala source files in a chiselWare Core shall conform to the Scala Style Guide:

`http://docs.scala-lang.org/style/`

Conformance to the Scala Style Guide is verified as part of the Static Compliance Checks defined in Section 6 using `scalafmt`. The `scalafmt` configuration file shall be the version distributed as part of the Standard Software Environment for the target Certification Version.

10.1.2 Chisel Coding Style

All Chisel source files in a chiselWare Core shall conform to the Chisel Developer's Style Guide:

`https://www.chisel-lang.org/docs/developers/style/`

Conformance to the Chisel Developer's Style Guide is verified as part of the Static Compliance Checks defined in Section 6 using `scalafix`. The `scalafix` configuration file shall be the version distributed as part of the Standard Software Environment for the target Certification Version.

10.2 chiselWare Coding Style

The following coding style requirements are specific to chiselWare and are verified by the Automated Design Review system.

10.2.1 Precedence

Where chiselWare coding style requirements are more specific than or supplementary to the Scala or Chisel style guides, the chiselWare requirements shall take precedence.

10.2.2 Scripting and Build Languages

The following scripting and build languages are permitted in a chiselWare Core repository:

Language	Purpose	Style Reference
Bash	All shell scripting	Google Shell Style Guide
Make	All build automation	GNU Make Conventions

No scripting or build language other than Bash and Make shall be introduced as a dependency of a chiselWare Core repository. This requirement is in addition to the SSE constraint defined in Section 7.

10.2.3 Hardware Description Languages

All synthesizable RTL should be generated exclusively from Chisel source. Hand-written Verilog or SystemVerilog blackbox definitions should be avoided except where required to describe hardware constructs that cannot be expressed in Chisel, such as technology-specific latches or asynchronous logic primitives. Where hand-written Verilog or SystemVerilog is needed, it shall be confined to the `src/main/resources/vsrc/` directory and shall be limited to blackbox definitions only.

The intent of this standard is that chiselWare Cores are designed in Chisel. Wrapping existing Verilog designs in a Chisel blackbox interface for the purpose of presenting them as chiselWare Cores is contrary to this intent and shall not be considered a legitimate path to chiselWare Certification.

10.2.4 Scaladoc Requirements

All public classes, methods, and parameters shall be documented with Scaladoc comments. The following rules apply:

1. Scaladoc comments shall include a description of the purpose of the class or method.
2. All parameters shall be documented using the `@param` tag.
3. Return values shall be documented using the `@return` tag where applicable.
4. The preferred documentation style is a narrative preamble that provides context for the code rather than inline comments that restate what the code does line by line.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 11

Verification

11.1 General

Verification is a mandatory deliverable for all chiselWare Cores. The verification process shall be approached with the same rigor as the design itself. All verification shall be executed within the Standard Software Environment defined by the chiselWare Administrator for the target Certification Version.

11.2 Directed Tests

Every chiselWare Core shall include a suite of self-checking directed tests that verify the functional correctness of the core. Directed tests shall be written in Scala using the ChiselTest framework distributed as part of the Standard Software Environment.

For cores with parameters, directed tests shall cover at minimum the default, maximum, and minimum configurations as defined in Section 9.

11.3 Constrained-Random Verification

Where possible, directed tests should incorporate constrained-random techniques to exercise random combinations of inputs, events, and timing beyond what deterministic directed tests alone can achieve. The goal is to increase confidence in functional correctness across the full input space of the core without requiring a separate constrained-random test infrastructure.

11.4 Coverage Requirements

Directed tests shall satisfy the following minimum coverage requirements as measured by Scoverage:

Metric	Minimum
Statement Coverage	95%
Branch Coverage	90%

11.4.1 Configuration Coverage

For cores with parameters, directed tests shall include tests for at least the following three configurations:

Configuration	Description
Default	All parameters at their default values as defined in the `<CoreName>Params` case class
Maximum	All parameters set to their maximum permitted values as defined by `require` constraints
Minimum	All parameters set to their minimum permitted values as defined by `require` constraints

Additional configurations may be defined at the discretion of the Core Developer.

11.4.2 Test Documentation

Each directed test shall include a Scaladoc comment specifying:

1. The purpose of the test
2. The configuration under which the test is executed
3. The expected behavior and the conditions under which the test shall pass or fail

11.5 Formal Verification

11.5.1 General

Formal verification is an optional but encouraged verification methodology for chiselWare Cores. It uses model checking to prove that specified properties hold for all possible input sequences up to a bounded depth, providing a higher level of assurance than simulation-based testing alone.

chiselWare formal verification uses the `chiseltest.formal` library in conjunction with the SymbiYosys (sby) formal verification front-end and the Z3 SMT solver, all distributed as part of the Standard Software Environment.

11.5.2 Formal Testbench Structure

Formal properties shall be defined in `<CoreName>FormalTb.scala` in the `src/test/` directory. The formal testbench follows the same pattern as the simulation testbench — it instantiates the core and adds properties — but uses `chiseltest.formal` primitives rather than ChiselTest simulation primitives.

The following temporal primitives are available via `import chiseltest.formal._`:

Primitive	Description
`past(sig)`	Value of `sig` one cycle ago (SVA: `$past`)
`rose(sig)`	True when `sig` transitions 0→1 (SVA: `$rose`)
`fell(sig)`	True when `sig` transitions 1→0 (SVA: `$fell`)
`stable(sig)`	True when `sig === past(sig)` (SVA: `$stable`)

Copyright © 2026 Rocksavage Technology, Inc.

Two important notes on temporal primitives:

- `reset` is of type `chisel3.Reset`, not `Bool` — always cast with `reset.asBool` before use in properties.
- For registered outputs, use `past(rose(sig))` and `past(fell(sig))` rather than `rose(sig)` and `fell(sig)` directly — the effect of a clock edge on a registered output is visible one cycle after the edge, not on the same cycle.

Three property types are available:

- `assert()` — a property that must hold for all reachable states. A failing assert indicates a bug.
- `assume()` — a constraint on the environment. Use to restrict inputs to legal combinations for the core being verified.
- `cover()` — a reachability check. A failing cover indicates that the specified state is unreachable, which may indicate over-constraining assumptions.

Copyright © 2026 Rocksavage Technology, Inc.

11.5.3 Formal Test Runner

The formal test runner in `<CoreName>FormalTest.scala` uses the `verify()` method with `BoundedCheck(k)` to run Bounded Model Checking (BMC) for `k` cycles after reset. The bound `k` shall be chosen to provide meaningful coverage of the core's behavior — deeper pipelines and more complex state machines require larger bounds.

Formal tests shall iterate over `simConfigMap` to verify properties across all parameter configurations, using the same pattern as the simulation test suite:

```
for ((configName, config) <- <CoreName>Params.simConfigMap) {
  it should s"pass all formal properties ($configName)"
      taggedAs FormalTest in {
    verify(
      new <CoreName>FormalTb(config),
      Seq(
        BoundedCheck(10),
        TargetDirAnnotation(
          s"modules/<corename>/generated/formal/$configName")
      )
    )
  }
}
```

The `FormalTest` tag defined in `<CoreName>FormalTest.scala` allows formal tests to be run independently at the sbt level:

```
sbt "testOnly * -- -n FormalTest"  # formal only
sbt "testOnly * -- -l FormalTest"  # exclude formal
```

Regression Integration

Formal tests run as part of `make test` and `make cov` alongside simulation tests. No separate make target is required. Results appear in `generated/test.rpt` alongside simulation results.

11.5.4 ADR Checking

The ADR system checks for the presence of `<CoreName>FormalTb.scala` and `<CoreName>FormalTest.scala`. If present, the ADR system evaluates their content for compliance with the property structure defined in this section. If absent, no finding is generated — formal verification is optional and the absence of formal files does not affect certification eligibility.

Core Developers who include the formal files from the `00-000-dff` template but do not implement formal verification for their core shall delete both files before certification submission. Empty or placeholder formal files constitute a conformance failure.

11.6 Code Coverage

11.6.1 General

All chiselWare Cores shall measure Scala code coverage using the version of Scoverage distributed as part of the Standard Software Environment. Coverage shall be measured across the combined

Copyright © 2026 Rocksavage Technology, Inc.

directed and constrained-random test suites in all configurations.

Tests shall satisfy the following minimum coverage requirements as measured by Scoverage:

Metric	Minimum
Statement Coverage	95%
Branch Coverage	90%

11.6.2 Coverage Reporting

A coverage report shall be generated as part of every regression run as defined in Section 13. The coverage report shall be retained as a required deliverable of the core release.

11.6.3 Coverage Exclusions

Where a Core Developer excludes specific code regions from coverage measurement, each exclusion shall be documented with a Scaladoc comment explaining the justification for the exclusion. Undocumented coverage exclusions shall constitute a conformance failure.

The chiselWare Administrator reserves the right to reject the Core Developer's justification for any coverage exclusion and require that the excluded code region be covered by additional tests. A coverage exclusion that is rejected by the chiselWare Administrator shall constitute a conformance failure until resolved to the Administrator's satisfaction.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 12

Documentation

12.1 General

Documentation is a mandatory deliverable for all chiselWare Cores and shall be treated with the same rigor as RTL and verification deliverables. No chiselWare Core shall be certified without complete documentation as defined in this section. The standard of documentation shall be such that a Core User can fully integrate and configure the core without reference to the core source code.

12.2 Scaladoc Requirements

12.2.1 General

All public classes, methods, and parameters in a chiselWare Core shall be documented with Scaladoc comments conforming to the requirements of Section 10.4.4. Scaladoc documentation shall be generated as part of every regression run as defined in Section 13.

12.2.2 Publication

Scaladoc generation is required to enable documentation publication within the chiselWare ecosystem and the IP Factory platform. chiselWare Cores are not required to be published to external package registries such as Maven Central. Publication of chiselWare Core artifacts to external registries is governed by the chiselWare Developer Agreement.

12.3 User Guide

12.3.1 General

Every chiselWare Core shall include a User Guide. The User Guide shall be authored in LaTeX using the template provided in the `00-000-dff` template repo for the target Certification Version. The User Guide shall be sufficient for a Core User to integrate, configure, simulate, and synthesize the core without reference to the core source code.

12.3.2 Required Sections

The User Guide shall contain at minimum the following sections in the order listed:

Section	Content
Version History	A table recording all released versions of the core, their release dates, and a description of changes
Errata and Known Issues	A list of known defects, limitations, and workarounds. Shall be present even if empty
Port Descriptions	A complete description of all input and output ports including name, direction, width, and functional description
Parameter Descriptions	A complete description of all parameters defined in the `<CoreName>Params` case class including name, type, default value, permitted range, and functional description
Simulation	Instructions for running the verification suite against the core, including the target frequency
Synthesis	Instructions for running synthesis and static timing analysis against the core, including the target frequency

12.3.3 Version Consistency

The version identifier recorded in the User Guide Version History shall exactly match the version recorded in `build.sbt` and the Git release tag as defined in Section 8. Any discrepancy between these three version identifiers shall constitute a conformance failure.

The date recorded in the User Guide Version History reflects the date on which the documentation was completed by the Core Developer and is not required to match the date of the Git release tag. The Git release tag date is determined by the certification process and will typically be later than the User Guide date.

12.3.4 Port Description Consistency

The port descriptions in the User Guide shall exactly match the ports defined in the top-level Chisel module. Any port present in the RTL and absent from the User Guide, or present in the User Guide and absent from the RTL, shall constitute a conformance failure.

12.3.5 Parameter Description Consistency

The parameter descriptions in the User Guide shall exactly match the parameters defined in the `<CoreName>Params` case class. The following shall each constitute a conformance failure:

1. A parameter present in the case class and absent from the User Guide.
2. A parameter present in the User Guide and absent from the case class.
3. A default value documented in the User Guide that does not match the default value defined in the case class.
4. A parameter bound documented in the User Guide that does not match the corresponding `require` constraint in the case class.

Copyright © 2026 Rocksavage Technology, Inc.

12.3.6 Generated Formats

The User Guide shall be generated in both PDF and HTML formats from the LaTeX source as part of every regression run defined in Section 13. Generated PDF and HTML files shall be retained as required deliverables of the core release.

12.4 README

12.4.1 General

Every chiselWare Core repository shall contain a `README.md` file at the repository root conforming to the template provided in the `00-000-dff`.

12.4.2 Required Content

The `README.md` shall contain at minimum the following:

1. The chiselWare Core name
2. The chiselWare Version that the core is certified to
3. The Organization ID and Team ID associated with the core
4. A version history
5. A description of the core and a link to its User Guide
6. How to generate Verilog
7. How to run a simulation
8. How to run synthesis
9. An authors section with the chiselWare Developer's GitHub user ID
10. A reference to the Apache2 license

12.5 License

12.5.1 Permitted License Terms

All chiselWare Cores shall be licensed under one of the following permitted licenses. No other license terms are permitted for chiselWare Certified cores:

- **Apache License, Version 2.0** — The standard open source license for chiselWare Cores made freely available to the public. The full text of the Apache License, Version 2.0 is available at:

  `https://www.apache.org/licenses/LICENSE-2.0`

- **chiselWare Government License (CW-GL)** — A license for cores developed under government or defense industrial base (DIB) funding. The CW-GL governs distribution rights, export control obligations, and government purpose rights in accordance with applicable federal acquisition regulations. The CW-GL is administered by the chiselWare Administrator.

Copyright © 2026 Rocksavage Technology, Inc.

- **chiselWare Commercial License (CW-CL)** — A proprietary license for cores distributed through the chiselWare IP Factory platform. The CW-CL governs commercial distribution, sublicensing, and royalty terms. The CW-CL is administered by the chiselWare Administrator and requires execution of a separate commercial agreement prior to certification.

The applicable license shall be recorded in the chiselWare Core Registry at the time of certification. A Core Developer may not change the license type of a certified core version after certification has been granted. A change of license type shall require submission of a new core version and recertification under the new license terms.

License File

Every chiselWare Core repository shall contain a `LICENSE.md` file at the repository root. The content of `LICENSE.md` shall depend on the applicable license type as follows.

For Apache 2.0 cores, `LICENSE.md` shall contain the full unmodified text of the Apache License, Version 2.0. No other content is permitted.

For CW-GL cores, `LICENSE.md` shall contain the SPDX license identifier followed by the contact details of the Organization through which license terms may be obtained:

```
SPDX-License-Identifier: LicenseRef-chiselWare-GL

License inquiries:
  <Organization name>
  <Contact name or role>
  <Email address>
  <URL (optional)>
```

For CW-CL cores, `LICENSE.md` shall contain the SPDX license identifier followed by the contact details of the Organization through which license terms may be obtained:

```
SPDX-License-Identifier: LicenseRef-chiselWare-CL

License inquiries:
  <Organization name>
  <Contact name or role>
  <Email address>
  <URL (optional)>
```

The contact details shall be those of the Organization registered in the chiselWare Core Registry through which license terms for the core may be obtained. This may be the Core Developer's Organization directly or the chiselWare Administrator acting as a licensing agent on behalf of the Core Developer. The chiselWare Core Registry is the authoritative source for contact details associated with each certified core.

The `LICENSE.md` file shall not contain any content that contradicts or modifies the terms of the license identified by the SPDX identifier. Any such contradiction shall constitute a conformance failure.

Copyright © 2026 Rocksavage Technology, Inc.

12.5.2 Source File Headers

Every Scala source file in a chiselWare Core shall begin with a copyright notice and SPDX license identifier header on the first two lines of the file without exception. The form of the header shall depend on the applicable license as follows.

For Apache 2.0 cores:

```
// Copyright (c) <year> <Core Developer or Organization name>
// SPDX-License-Identifier: Apache-2.0
```

For CW-GL cores:

```
// Copyright (c) <year> <Core Developer or Organization name>
// SPDX-License-Identifier: LicenseRef-chiselWare-GL
```

For CW-CL cores:

```
// Copyright (c) <year> <Core Developer or Organization name>
// SPDX-License-Identifier: LicenseRef-chiselWare-CL
```

The `LicenseRef-` prefix is the SPDX convention for licenses that are not on the SPDX standard license list. The identifiers `LicenseRef-chiselWare-GL` and `LicenseRef-chiselWare-CL` are registered and administered by the chiselWare Administrator.

The copyright year shall reflect the year of first publication of the file. The copyright holder shall be the Core Developer or their affiliated Organization as registered in the chiselWare Core Registry.

12.5.3 License Consistency

The following consistency requirements shall apply to all chiselWare Cores regardless of the applicable license type:

1. The license type identified in `LICENSE.md` shall exactly match the license type recorded in the chiselWare Core Registry for that core. Any discrepancy shall constitute a conformance failure.
2. The SPDX license identifier in every Scala source file header shall exactly match the license type recorded in the chiselWare Core Registry. Any discrepancy shall constitute a conformance failure.
3. For Apache 2.0 cores, the license text in `LICENSE.md` shall be the unmodified text of the Apache License, Version 2.0. Any modification to the license text shall constitute a conformance failure.
4. For CW-GL and CW-CL cores, `LICENSE.md` shall not contain any content that contradicts or modifies the terms of the license identified by the SPDX identifier. Any such contradiction shall constitute a conformance failure.
5. A Core Developer may not change the license type of a certified core version after certification has been granted. A change of license type shall require submission of a new core version and recertification under the new license terms.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 13

Regressions and CI/CD

13.1 General

Every chiselWare Core shall include a regression harness that automatically executes the full complement of verification, synthesis, and documentation generation tasks defined in this standard. The regression harness is defined by the `Makefile` and GitHub Actions workflow at the repository root. Both are provided as templates in the `00-000-dff` template repository and require only minimal customization by the Core Developer, specifically the `CORE_NAME` and `CORE_DIR` variables at the top of the `Makefile`. All regression tasks shall execute exclusively within the Standard Software Environment defined in Annex D.

13.2 Makefile

The `Makefile` at the repository root is the single source of truth for regression behavior. The primary regression entrypoint shall be:

```
make all
```

The `make all` target shall execute the complete regression sequence defined in Section 13.3. The same `make all` entrypoint is used for both local development and CI regression runs, ensuring behavioral consistency between the two environments.

13.2.1 Required Make Targets

The `Makefile` shall define exactly the following targets and no others:

Target	Description
`all`	Execute the complete regression sequence
`check`	Scan all generated reports for errors and generate `error.rpt`
`clean`	Remove all generated artifacts and intermediate files
`cov`	Execute all tests with Scoverage code coverage enabled and generate coverage report
`docs`	Generate Scaladoc API documentation and User Guide PDF
`ipf`	Generate IP Factory deliverables in the `.ipf/` directory
`lint`	Execute `scalafmt` formatting check and `scalafix` lint checks
`list`	List all available Makefile targets
`publish`	Publish core libraries locally via `publishLocal`
`test`	Execute all tests without coverage instrumentation
`verilog`	Generate Verilog from Chisel source
`yosys`	Execute synthesis and static timing analysis on generated Verilog

13.3 Regression Sequence

The `make all` target shall execute the following tasks in the order listed. All tasks shall complete without error before the regression is considered passing:

Step	Target	Description
1	`clean`	Remove all generated artifacts
2	`lint`	`scalafmt` and `scalafix` checks
3	`publish`	Publish libraries locally
4	`cov`	Run tests with coverage enabled
5	`yosys`	Generate Verilog and run synthesis
6	`docs`	Generate Scaladoc and User Guide PDF
7	`ipf`	Generate IP Factory deliverables
8	`check`	Scan reports and generate `error.rpt`

13.4 Regression Pass/Fail Criterion

The authoritative pass/fail criterion for every regression run is the error report generated by the `check` target at the following path:

```
modules/<module_name>/generated/error.rpt
```

The `check` target scans the following reports for errors and aggregates all findings into `error.rpt`:

Copyright © 2026 Rocksavage Technology, Inc.

Report	Checks Applied
`lint.rpt`	`scalafmt` and `scalafix` errors
`docs/doc.rpt`	All errors
`docs/publish.rpt`	All errors and warnings
`generated/verilog.rpt`	All errors
`generated/test.rpt`	All errors and failures
`synTestCases/area_summary.rpt`	All errors
`synTestCases/timing_summary.rpt`	All errors
`synTestCases/*/timing.rpt`	All errors
`synTestCases/*/yosys.log`	All errors
`.ipf/` directory	Presence of all required IP Factory deliverables

The `error.rpt` file shall be interpreted as follows:

Condition	Result
File absent	Regression failed
File present and non-empty	Regression failed; contents identify the cause
File present and empty	Regression passed

13.5 Synthesis Requirements

13.5.1 General

Every chiselWare Core shall include a synthesis framework conforming to the `00-000-dff` template. The `yosys` target shall first invoke the `verilog` target to generate Verilog from Chisel source, then execute synthesis and static timing analysis using the Yosys toolchain distributed as part of the Standard Software Environment.

13.5.2 Required Configurations

Synthesis shall be executed for at minimum the following three configurations as defined in Section 11.4.1:

1. The default configuration
2. The maximum configuration
3. The minimum configuration

Synthesis test cases shall be defined in `modules/<corename>/generated/synTestCases/` and executed via the `run.sh` script generated by the core.

13.5.3 Synthesis Deliverables

The following deliverables shall be generated and retained for each synthesized configuration:

Copyright © 2026 Rocksavage Technology, Inc.

Deliverable	Path	Description
Synthesis Log	`synTestCases/<config>/yosys.log`	Complete Yosys synthesis log
Timing Report	`synTestCases/<config>/timing.rpt`	Static timing analysis report
Area Summary	`synTestCases/area_summary.rpt`	Gate count and area utilization across all configurations
Timing Summary	`synTestCases/timing_summary.rpt`	Timing closure summary across all configurations

13.5.4 Timing Closure

A chiselWare Core shall achieve timing closure at the target frequency defined in the User Guide Synthesis section for all required synthesis configurations. Failure to achieve timing closure shall constitute a conformance failure.

13.6 IP Factory Deliverables

13.6.1 General

Every chiselWare Core shall generate a complete set of IP Factory deliverables as part of every regression run. IP Factory deliverables are generated by the `ipf` target and placed in the `.ipf/` directory at the repository root.

13.6.2 Required IP Factory Deliverables

The following files shall be present in the `.ipf/` directory upon successful completion of the `ipf` target. Absence of any required deliverable shall constitute a conformance failure as verified by the `check` target:

File	Description
`<CoreName>.v` or `<CoreName>.sv`	Generated Verilog or SystemVerilog netlist
`<CoreName>.json`	Core metadata in JSON format
`<CoreName>.pdf`	User Guide PDF
`<CoreName>.sdc`	Synopsys Design Constraints file
`filelist.f`	Verilog file list

13.7 GitHub Actions Workflow

Every chiselWare Core repository shall include a GitHub Actions workflow at `.github/workflows/` that invokes the shared CI workflow framework maintained by

Copyright © 2026 Rocksavage Technology, Inc.

the chiselWare Administrator. The workflow is provided as a template in the `00-000-dff` template repository and shall not be modified by the Core Developer except to set the `module_name` input to the name of the core being developed.

The workflow shall be triggered on the following events:

1. Every pull request targeting `staging`
2. Manual invocation via `workflow_dispatch`

The only required customization to the workflow template is the `module_name` input, which shall be set to the lowercase core name matching the module directory name defined in Section 7.

Copyright © 2026 Rocksavage Technology, Inc.

Chapter 14

Licensing and Provenance

14.1 Scope

This section establishes normative requirements governing the origin, licensing, and attribution of source code incorporated into a chiselWare Core. These requirements exist to protect developers, downstream integrators, and the chiselWare ecosystem from license incompatibility, attribution failure, and unenforceable or unknown-provenance code.

14.2 Policy of Original Authorship

A chiselWare Core SHALL consist of code that is either:

1. original work authored by the contributing developer(s), or
2. incorporated from a third-party source that satisfies all conditions in Section 14.4 of this chapter.

Incorporation of third-party code is the exception, not the default path for chiselWare development. Developers SHOULD prefer writing original implementations over incorporating external code, even when a permissively-licensed alternative exists.

14.3 Prohibited Provenance

A chiselWare Core SHALL NOT incorporate source code or documentation text from any of the following:

1. Any file, repository, or snippet that does not carry an explicit, identifiable license. The absence of a license notice SHALL be treated as "all rights reserved" and therefore as granting no permission to copy, modify, or redistribute, regardless of the source's public availability.
2. Any source licensed under a strongly-reciprocal ("copyleft") license, including but not limited to CERN-OHL-S, GPL (any version), and AGPL.Such licenses impose licensing obligations on the resulting work that are incompatible with chiselWare's Apache 2.0 licensing model.
3. Any source licensed under terms that impose field-of-use restrictions, non-commercial-use restrictions, or patent-retaliation clauses broader than those in Apache 2.0.

4. Any source whose license terms cannot be confidently identified by the contributing developer. When in doubt, the source SHALL be treated as prohibited under this section.

14.4 Permitted Provenance

Third-party source code MAY be incorporated into a chiselWare Core only if it is licensed under the Apache License, Version 2.0. No other license is permitted, regardless of compatibility or permissiveness. chiselWare is exclusively Apache 2.0; this applies uniformly to all source code, scripts, and other content within a Core, with no separate allowance for any other license.

14.5 Attribution Requirements

When third-party code is incorporated under Section 14.4, the contributing developer SHALL:

1. Preserve the original copyright notice and license text in full. The original notice SHALL NOT be removed, replaced, or overwritten.
2. Add a separate copyright line for any new contributions, rather than altering the original author's copyright line.
3. Identify the provenance of the incorporated code (origin repository or project, and the version or commit referenced) in a comment adjacent to the preserved notice.
4. Where the upstream source repository carries a `NOTICE` file or other attribution content, carry the relevant attribution into this Core's `AUTHORSHIP.md`, per the chiselWare Developers Guide.
5. Where the incorporated source originates from a repository that is not uniformly licensed, confirm that the specific file or module incorporated is itself attributable to an Apache 2.0 origin, per the guidance in the chiselWare Developers Guide.

Practical templates and a worked example satisfying this section are provided in the chiselWare Developers Guide.

14.6 Relationship to Certification

Conformance with this section is a precondition for chiselWare certification. Static Compliance Checks SHALL verify that every source file under `src/` and `test/` carries either an original chiselWare copyright notice or a properly preserved third-party notice consistent with Section 14.5. A Core containing code of unidentifiable or prohibited provenance SHALL NOT be certified until the offending code is removed or replaced.

Copyright © 2026 Rocksavage Technology, Inc.

Annexes

Appendix A

Conformance Checklist (Normative)

A.1 General

This annex defines every normative conformance requirement of this standard and specifies the mechanism by which each requirement is verified. Core Developers shall use this annex as the authoritative reference for understanding what is required for chiselWare Certification and how each requirement will be checked. The rule identifiers used in this annex correspond directly to the rule identifiers reported in the chiselWare Automated Design Review compliance report.

Each requirement specifies the verification mechanism as follows:

Code	Mechanism	Description
ADR	Automated Design Review	Verified automatically by the ADR rule engine
SCC	Static Compliance Check	Verified by `scalafmt` or `scalafix`
CI	CI Workflow	Verified by the regression harness and `error.rpt`
MAN	Manual Review	Verified during the certification process by the chiselWare Administrator

A.2 Scala Coding Style

All Scala coding style rules are verified by Static Compliance Check (SCC) using the `scalafmt` and `scalafix` configurations distributed as part of the Standard Software Environment. The SSE configurations are authoritative and cannot be overridden by Core Developer local settings.

A.2.1 SCALA-001

Line length does not exceed 80 characters.

All Scala source lines shall be 80 characters or fewer. This limit is enforced by SCC.

A.2.2 SCALA-002

Line length does not exceed 80 characters.

A method invocation whose arguments do not fit within 80 characters shall be wrapped, with each continuation line indented two spaces from the line containing the invocation. This rule is enforced by SCC.

A.2.3 SCALA-019

Curly braces are properly nested

Parentheses shall not be separated from their contents by spaces. An opening parenthesis shall be kept on the same line as the start of its contents. When the contents do not fit within the line-length limit, they may be broken across lines as determined by the formatter, with the closing parenthesis placed on its own line. This rule is enforced by SCC.

A.2.4 SCALA-020

Parentheses are properly spaced

Parentheses shall not be separated from their contents by spaces. An opening parenthesis shall be kept on the same line as the start of its contents. When the contents do not fit within the line-length limit, they may be broken across lines as determined by the formatter, with the closing parenthesis placed on its own line. This rule is enforced by SCC.

A.2.5 SCALA-021

Classes are compactly declared

A class constructor shall be declared on a single line when the declaration fits within 80 characters. Otherwise, each parameter shall be placed on its own line, followed by a trailing comma, with the closing parenthesis on a separate line. This rule is enforced by SCC.

A.2.6 SCALA-022

Class elements are well-ordered

Members of a class, object, or trait that span multiple lines shall be separated from adjacent members by a single empty line. Consecutive single-line declarations, such as related signal or register definitions, may be grouped without intervening empty lines. This rule is enforced by SCC.

A.2.7 SCALA-023

Functions should specify a return type

Every method defined as a member of a class, object, or trait shall declare an explicit result type, including methods returning `Unit`. This rule is enforced by SCC.

Copyright © 2026 Rocksavage Technology, Inc.

A.2.8 SCALA-024

Parentheses and curly braces are properly spaced

There shall be no space between parentheses and the code they contain. Curly braces shall have a single space of separation from the code they enclose, except within string interpolations. This rule is enforced by SCC.

A.2.9 SCALA-025

Control structure keywords are followed by a space

Scala control-structure keywords (`if`, `while`, `for`, `match`, `catch`) shall be followed by a single space before an opening parenthesis. Chisel hardware constructs such as `when`, `elsewhen`, `switch`, and `is` are method calls, not keywords, and shall be written without a space. This rule is enforced by the Scala Compiler and by SCC.

A.2.10 SCALA-026

Curly braces are used appropriately in control structures

Curly braces shall not be omitted from `while` loops or `match` expressions. Braces may be omitted from `if` expressions whose branches are single-line, pure expressions. Chisel constructs such as `when` take their body as a block argument and are not subject to this rule. This rule is enforced by the Scala compiler and by SCC.

A.2.11 SCALA-029

Files should contain only one class

Each source file shall contain at most one class definition, and a file containing a class shall be named after that class; for example, class `Foo` shall be defined in `Foo.scala`. A file may additionally contain any number of objects, traits, and case classes. This rule is enforced by SCC.

A.3 Chisel Coding Style

All Chisel coding style rules are verified by the ADR rule engine. These rules extend the Scala coding style requirements with Chisel-specific conventions defined in the Chisel Developer's Style Guide and the chiselWare-specific clarifications defined in Section 10.2.

A.3.1 CHISEL-001

Scala file name matches the top-level class it contains.

Each Scala source file shall be named after the top-level class it contains, with the `.scala` extension. For example, a file containing the class `MyCore` shall be named `MyCore.scala`. This requirement is in addition to the Scala Style Guide convention and is verified by the ADR rule engine by cross-referencing file names against class declarations.

Copyright © 2026 Rocksavage Technology, Inc.

A.3.2 CHISEL-002

Design-Specific Language (DSL) imports should be first.

Each Scala source file shall have its DSL imports be listed first. Examples of there are `chisel._` and textttchiseltest._. This requirement is enforced by SCC.

A.3.3 CHISEL-003

DSL imports should be separated from any following imports by a blank line.

Each Scala source file shall have its DSL imports be listed first. Examples of there are `chisel._` and textttchiseltest._. This requirement is enforced by SCC.

A.3.4 CHISEL-004

Imports should be listed alphabetically.

Each Scala source file shall have its imports listed alphabetically. This requirement is enforced by SCC.

A.3.5 CHISEL-005

Imports should not use wildcards.

Except for DSL imports, imports should not use wild cards. This requirement is enforced by SCC.

A.4 chiselWare Guidelines

chiselWare guideline rules verify conformance to the structural and organizational requirements specific to chiselWare Cores as defined in Sections 7 through 9 of this standard. All rules in this category are verified by the ADR rule engine.

A.4.1 CW-001

All required Scala and Chisel files are present.

The repository shall contain all required source files in the locations defined in Section 7. This includes the top-level module file, the parameter case class file, and the test suite file. The ADR system verifies the presence of each required file by name and location. Missing files will be reported individually in the compliance report. Developers should initialize their repository from the `00-000-dff` template to ensure all required files are present from the outset.

A.4.2 CW-007

Required template files present.

Every core repository shall contain the template files that the Template Conformance Check examines: `.scalafmt.conf`, `.scalafix.conf`, `.scalafix-test.conf`, `project/plugins.sbt`, `build.sbt`, and `Makefile`. Each file shall be present at the same path as in the chiselWare template repository (`00-000-dff`). A missing file constitutes a

Copyright © 2026 Rocksavage Technology, Inc.

conformance failure. The Template Conformance Check runs locally as `make conformance`, and in CI before the ADR system is invoked.

A.4.3 CW-008

Tooling configuration files unmodified.

The files `.scalafmt.conf`, `.scalafix.conf`, `.scalafix-test.conf`, and `project/plugins.sbt` shall be identical to the corresponding files in the chiselWare template release that matches the core's `chiselWareVersion`. These files define the formatting, linting, and build plugins on which other rules depend, and they shall not be modified by the core developer. The comparison ignores differences in line endings, trailing whitespace, and trailing blank lines; any other difference constitutes a conformance failure. The check reports a unified diff against the template to show the lines that differ.

A.4.4 CW-009

Configuration block markers.

The files `build.sbt` and `Makefile` shall each contain a single Core Configuration block, delimited by the begin and end marker lines copied unchanged from the chiselWare template. Each marker line shall appear exactly once, and the begin marker shall precede the end marker. A missing, duplicated, altered, or misordered marker line constitutes a conformance failure, since the remaining checks on the file cannot be performed without it.

A.4.5 CW-010

Content outside the configuration block.

In `build.sbt` and `Makefile`, all lines above the begin marker and below the end marker shall be identical to the corresponding lines of the chiselWare template release that matches the core's `chiselWareVersion`. Core-specific settings belong only inside the Core Configuration block. The comparison ignores differences in line endings, trailing whitespace, and trailing blank lines; any other difference, including a change from tab to space indentation in a `Makefile` recipe, constitutes a conformance failure. The check reports a unified diff against the template to show the lines that differ.

A.4.6 CW-011

Configuration block entries.

The Core Configuration block shall contain each of the entries listed in Table A.1 exactly once, and no other entries. Each value shall be a literal: a quoted string, or for `spdxCopyrightText` a `Seq` of quoted strings. Expressions, references to other values, and function calls are not permitted. Blank lines and comments are permitted, and an entry may span several lines when wrapped by the formatter. An unknown, missing, or duplicated entry, or a value that does not conform to its required format, constitutes a conformance failure.

A.4.7 CW-012

Consistency between `build.sbt` and `Makefile`.

Copyright © 2026 Rocksavage Technology, Inc.

Table A.1: Core Configuration block entries (CW-011).

File	Entry	Required format
`build.sbt`	`coreNameLc`	Lowercase letter, then lowercase letters or digits
	`coreVersion`	Semantic version *x.y.z*
	`orgId`	Two digits
	`teamId`	Three digits
	`chiselWareVersion`	Semantic version *x.y.z*
	`spdxCopyrightText`	`Seq` of one or more non-empty strings
`Makefile`	`ORG_ID`	Two digits
	`TEAM_ID`	Three digits
	`CORE_NAME`	Uppercase letter, then letters or digits
	`IPF_PARAMS`	Comma-separated list of *name=value* pairs

Values that appear in both Core Configuration blocks shall agree. `orgId` shall equal `ORG_ID`, `teamId` shall equal `TEAM_ID`, and `coreNameLc` shall equal `CORE_NAME` converted to lowercase. A disagreement constitutes a conformance failure. This rule detects the common error of renaming or renumbering a core in one file but not the other.

A.4.8 CW-013

Platform version.

The `chiselWareVersion` entry in `build.sbt` shall equal the version of the Template Conformance Check that performs the check. Because `make conformance` selects the checker version from `chiselWareVersion`, this rule ensures that the core is compared against the template release that it declares. A mismatch constitutes a conformance failure.

A.5 Code Coverage

Code coverage requirements are verified by the CI workflow using the Scoverage report generated by the `cov` target. Coverage is measured across the combined directed and constrained-random test suites.

A.5.1 COV-001

Code coverage meets minimum requirements.

The combined test suite shall achieve a minimum of 95% statement coverage and 90% branch coverage as measured by Scoverage. The coverage report is generated by the `cov` target and results below the minimum thresholds will appear in `error.rpt` and constitute a regression failure. If specific code regions are intentionally excluded from coverage measurement each exclusion shall be documented per CW-011.

Copyright © 2026 Rocksavage Technology, Inc.

A.6 Synthesis Deliverables

Synthesis deliverable requirements are verified by a combination of ADR and CI checks. The ADR system verifies the structure and content of synthesis scripts and configuration files. The CI workflow verifies that synthesis completes successfully and that all required output files are generated.

A.6.1 SYN-001

Required synthesis files are present.

The `synTestCases` directory shall contain at least one subdirectory, a `run.sh` script, an `area_summary.rpt` file, and a `timing_summary.rpt` file. The ADR system verifies the presence of each required file. Missing files are reported individually in the compliance report.

A.6.2 SYN-002

At least three synthesis configurations are present.

The `synTestCases` directory shall contain at least three subdirectories corresponding to the default, maximum, and minimum parameter configurations defined in Section 13.5.3. Additional configurations may be present. The ADR system counts the number of subdirectories and flags a failure if fewer than three are present.

A.6.3 SYN-009

Timing reports are consistent and timing closure is achieved.

The `timing_summary.rpt` file shall be consistent with the individual `timing.rpt` files in each synthesis configuration subdirectory. The CI workflow verifies that timing closure is achieved at the target frequency defined in the User Guide for all required synthesis configurations. Timing closure failures are reported in `error.rpt` with the affected configuration identified.

A.7 Documentation

Documentation requirements are verified by the ADR rule engine, which parses both the LaTeX source of the User Guide and the Scala source of the core to verify consistency between the two.

A.7.1 DOC-001

Required documentation files are present.

The `docs/user-guide/` directory shall contain all required files as defined in Section 7.7. The ADR system verifies the presence of each required file including the top-level LaTeX source file, the `ports.tex`, `params.tex`, `sim.tex`, `syn.tex`, and `errata.tex` section files, and the `images/` directory. Missing files are reported individually.

A.7.2 DOC-002

Documentation structure is correct and complete.

Copyright © 2026 Rocksavage Technology, Inc.

The User Guide shall contain all required sections in the order defined in Section 12.3.2. The ADR system verifies that each required section is present and contains content. The title page shall include the core name, version, a GitHub repository URL, and the author name. Empty sections constitute a conformance failure.

A.7.3 DOC-003

Port list documentation is complete and consistent with RTL.

The port descriptions in `ports.tex` shall exactly match the ports defined in the top-level Chisel module. Port names and directions are checked case-sensitively. The `clock` and `reset` ports are exempt from this check. Any port present in the RTL and absent from the documentation, or present in the documentation and absent from the RTL, constitutes a conformance failure.

A.7.4 DOC-004

Port width specifications are consistent between RTL and documentation.

The width of each port documented in `ports.tex` shall match the width defined in the top-level Chisel module. Widths may be expressed as integer literals or as expressions involving parameters such as `log2Ceil(parameter)`. The ADR system evaluates parameter expressions using the default parameter values defined in the `<CoreName>Params` case class.

A.7.5 DOC-005

Grammar check.

The LaTeX source files of the User Guide shall be free of grammatical and spelling errors. The ADR system performs a grammar and spelling check on all `.tex` files. Minor stylistic suggestions are reported as warnings rather than failures. Significant grammatical errors that impair understanding constitute a conformance failure at the discretion of the chiselWare Administrator.

A.7.6 DOC-006

Port descriptions are adequate in length and content.

Each port description in `ports.tex` shall be between 4 and 256 characters in length and shall provide a meaningful functional description of the port. Descriptions that consist solely of the port name restated or that are otherwise uninformative constitute a conformance failure at the discretion of the chiselWare Administrator.

Copyright © 2026 Rocksavage Technology, Inc.

Appendix B

Relationship to the chiselWare Developers Guide (Informative)

This annex is informative. It does not contain normative requirements.

The chiselWare Developers Guide (CWDG) is the companion document to this standard. Where this standard defines what is required for chiselWare Certification, the CWDG explains why requirements exist, provides practical guidance on meeting them, and describes the tools and workflows that make compliance achievable.

Topic	CWS	CWDG
Normative requirements	✓	—
Conformance checklist	✓	—
ADR rule definitions	✓	—
Philosophy and motivation	—	✓
Dev Kit walkthrough	—	✓
CI/CD architecture detail	—	✓
Runner configuration	—	✓
Security model	—	✓
Worked examples	—	✓
Certification submission workflow	—	✓

In the event of any conflict between the CWDG and this standard, this standard shall take precedence.

Appendix C

Standard Software Environment (Normative)

C.1 General

The Standard Software Environment (SSE) defines the complete, version-pinned toolchain against which all chiselWare Cores shall be developed, verified, synthesized, and documented. The SSE is defined by the toolchain specification in Section C.3, which is the authoritative reference for conformance purposes. Use of any tool version or combination not defined in this annex shall constitute a conformance failure.

The SSE is versioned synchronously with this standard. Each version of the CWS defines exactly one SSE. A new version of the CWS shall always define a new SSE version.

Core Developers and organizations may deploy the SSE using any of the following methods, all of which are officially supported by the chiselWare Administrator:

- **Docker image** — A pre-built, version-pinned Docker image distributed via the chiselWare container registry. Recommended for individual developers and CI/CC.pipelines.
- **Azure Virtual Machine** — A pre-configured VM image available via the Azure Marketplace. Recommended for teams requiring a managed cloud development environment.
- **Self-built environment** — An environment constructed by the Core Developer or their organization directly from the toolchain specification in Section C.3. Recommended for organizations with internal IT policies that restrict the use of external Docker images or cloud services, such as defense contractors and large semiconductor companies.

Regardless of the deployment method used, the installed tool versions shall exactly match those specified in Section C.3. The chiselWare Administrator provides the Docker image and Azure VM as a convenience — the toolchain specification is the normative reference, not the distributed images.

C.2 Rationale

Version incompatibility is a known and significant risk in the Chisel ecosystem. Differences in Chisel, firtool, and ChiselTest versions frequently produce incompatible results and build failures. The chiselWare SSE adopts the same approach used by CAD departments in commercial EDA environments, where a single authoritative toolchain configuration is mandated across all projects to ensure reproducibility and eliminate version-driven inconsistencies.

Core Developers shall not install, substitute, or upgrade any toolchain component outside of those specified in Section C.3 for the target Certification Version.

C.3 Toolchain Specification

The following toolchain components are pinned at the versions listed for this release of CWS. This specification is the authoritative reference for all SSE deployment methods:

Component	Version	Purpose
OpenJDK	21.0.x	Java runtime for Scala and sbt
sbt	1.11.7	Scala build tool
Scala	2.13.13	Scala compiler
Scala CLI	1.10.1	Scala CLI
Chisel	5.3.0	Hardware description language
firtool	1.47.0	FIRRTL compiler and Verilog generator
ChiselTest	5.0.2	Simulation and test framework
Verilator	5.020-1	Open-source RTL simulator (compiled)
Icarus Verilog	12-0-2build2	Open-source RTL simulator (interpreted)
GTKWave	3.3.116-1build2	Waveform viewer
Yosys	0.33-5build2	Open-source synthesis tool
CUDA	3.0.0	Binary decision diagram library
OpenSTA	2.7.0 @aa3088885f	Static timing analysis
sby	v0.65	SymbiYosys formal verification frontend
Yices	2.6.4	SMT solver
Z3	Ubuntu 24.04 packaged	SMT solver
scalafmt	2.5.0	Scala code formatter
scalafix	0.13.0	Scala lint and rewrite tool
Scoverage	2.0.11	Scala code coverage tool
TeXLive	3.141592653-2.6-1.40.25	Documentation generation

The chiselWare Administrator provides the following pre-built distributions of the SSE as a convenience to Core Developers. These distributions are built directly from the toolchain specification in Section C.3 and are provided without warranty beyond their conformance to that specification.

C.3.1 Docker Image

A pre-built Docker image is publicly available at:

```
ghcr.io/chiselware/dev-full:1.0.0
```

Copyright © 2026 Rocksavage Technology, Inc.

The Dockerfile used to build this image is published in the chiselWare Dev Kit repository and is itself a required deliverable of each CWS release, enabling organizations to audit and reproduce the image independently.

C.3.2 Azure Virtual Machine

A pre-configured VM image is available via the Azure Marketplace under the chiselWare publisher account. The VM image provides the complete SSE toolchain in a managed cloud environment suitable for team-based development.

C.4 Self-Built Environments

Organizations that construct their own SSE environment from the toolchain specification in Section C.3 shall ensure that:

1. All tool versions exactly match those specified in Section C.3.
2. The build environment is isolated from tools not listed in Section C.3 to prevent unintended interactions.
3. The organization maintains records of their environment configuration sufficient to reproduce it and to demonstrate conformance to Section C.3 upon request by the chiselWare Administrator.

The chiselWare Administrator may request evidence of toolchain conformance as part of the certification process for cores submitted from self-built environments.

C.5 SSE Updates

The SSE is updated with each new version of this standard. The following rules apply:

1. A CWS Major or Minor version increment shall always produce a new SSE version with updated tool version pins.
2. A CWS Patch version increment shall not change the SSE version unless a critical security or correctness issue in a pinned tool requires an immediate update.
3. In the event of a Patch-level SSE update, the chiselWare Administrator shall notify all registered Core Developers and publish an errata notice against the affected CWS version.

Copyright © 2026 Rocksavage Technology, Inc.